

Université Virtuelle du Burkina Faso (UV-BF)



Rapport du Projet Tutoré

Application de gestion de pharmacie en ligne

Étudiant : Kassongo Moussa

Filière : Génie Logiciel – Développement Web et Mobile

Niveau : Licence 3

Tuteur : M. OUEDRAOGO W. A. Marc Christian

Année académique : 2023–2024

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectifs du projet	3
1.3	Description générale de la plateforme	3
2	Analyse des besoins	4
2.1	Fonctionnalités principales	4
2.2	Analyse des utilisateurs cibles	4
3	Conception et architecture	5
3.1	Choix des technologies	5
3.2	Architecture du système	5
4	Implémentation	6
4.1	Captures d'écran	6
4.1.1	Tableau de bord pharmacie	6
4.1.2	Catalogue et recherche	6
4.1.3	Panier et paiement	7
4.1.4	Écran de couverture	7
5	Tests et validation	8
5.1	Méthodologie	8
5.2	Tests des endpoints API	8
5.3	Tests de l'application Flutter	8

6 Performances et sécurité	9
6.1 Performances	9
6.2 Sécurité	9
6.3 Lien GitHub	9
7 Conclusion	10
Annexes	11

1. Introduction

1.1 Contexte du projet

Avec la croissance des technologies numériques, la gestion des activités commerciales tend à se digitaliser. Les pharmacies au Burkina Faso rencontrent encore des difficultés liées à la gestion manuelle des stocks, la disponibilité des médicaments, ainsi que l'organisation des commandes et des livraisons. Dans ce contexte, la création d'une solution numérique adaptée permettrait de moderniser la gestion pharmaceutique, rendre l'accès aux médicaments plus rapide et optimiser les opérations des pharmacies.

1.2 Objectifs du projet

L'objectif général de ce projet est de concevoir et implémenter une plateforme modernisée permettant la gestion des stocks et la commande de médicaments en ligne.

Les objectifs spécifiques sont :

- Faciliter l'inscription et la connexion des utilisateurs et pharmacies.
- Permettre la recherche de médicaments par nom, catégorie ou symptôme.
- Gérer les commandes en ligne et proposer un paiement sécurisé.
- Assurer le suivi des commandes et livraisons.
- Optimiser la gestion des prescriptions et des stocks.

1.3 Description générale de la plateforme

La plateforme se compose d'une application mobile destinée aux clients pour rechercher et commander des médicaments, et d'un tableau de bord pour permettre aux pharmacies de gérer leurs stocks et commandes. Le système communique avec une API développée avec Django permettant l'échange sécurisé des données avec la base PostgreSQL.

2. Analyse des besoins

2.1 Fonctionnalités principales

Les fonctionnalités clés identifiées sont :

- Gestion des comptes utilisateurs et pharmacies.
- Catalogue des médicaments.
- Recherche avancée.
- Panier et commande.
- Paiement sécurisé.
- Suivi des livraisons.
- Gestion des prescriptions.
- Gestion de stock.

2.2 Analyse des utilisateurs cibles

Les utilisateurs sont :

- **Clients** : recherchent, commandent et suivent leurs médicaments.
- **Pharmacies** : gèrent les stocks, disponibilités, commandes et livraisons.
- **Administrateur** : supervise le système global.

3. Conception et architecture

3.1 Choix des technologies

- **Backend** : Django (Python) — robuste et adapté pour les APIs via Django REST Framework.
- **Frontend** : Flutter — multiplateforme, rapide, moderne.
- **Base de données** : PostgreSQL.

3.2 Architecture du système

La plateforme suit une architecture client-serveur :

- Application Flutter (client),
- API REST Django (serveur),
- Base PostgreSQL.

4. Implémentation

4.1 Captures d'écran

4.1.1 Tableau de bord pharmacie

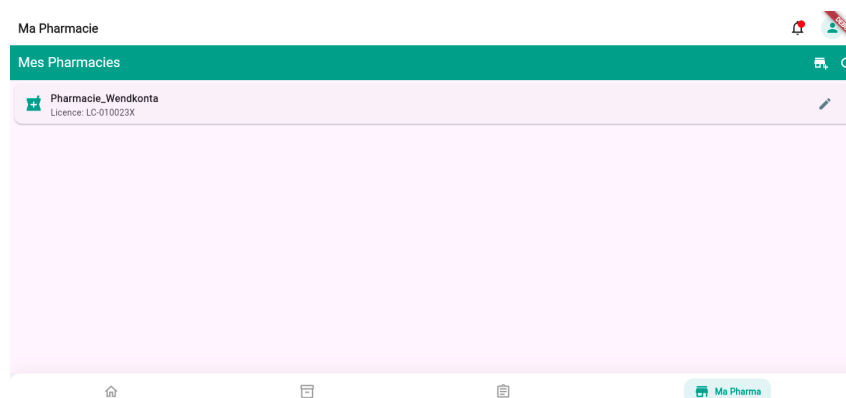


FIGURE 4.1 – Tableau de bord Pharmacie

4.1.2 Catalogue et recherche



FIGURE 4.2 – Catalogue des médicaments

4.1.3 Panier et paiement

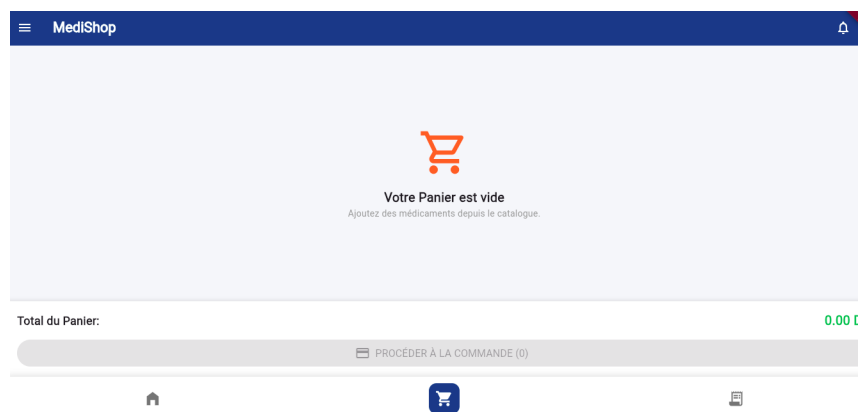


FIGURE 4.3 – Panier et processus de paiement

4.1.4 Écran de couverture

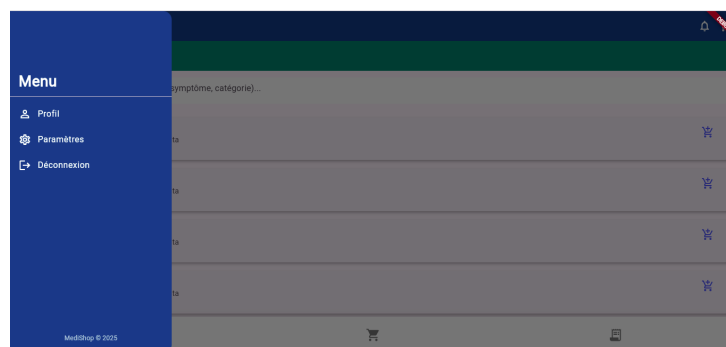


FIGURE 4.4 – Écran de couverture de l'application Medishop

5. Tests et validation

5.1 Méthodologie

Plusieurs types de tests ont été réalisés pour assurer la stabilité, la fiabilité et la cohérence de la plateforme.

5.2 Tests des endpoints API

Les endpoints Django REST Framework ont été testés à travers :

- **Tests unitaires** : validation du fonctionnement isolé des vues et serializers.
- **Tests fonctionnels** : vérification des opérations (création de comptes, commande, etc.).
- **Tests d'intégration** : test des interactions entre API, modèles et base PostgreSQL.

5.3 Tests de l'application Flutter

Les tests suivants ont été effectués :

- **Tests des interfaces** : navigation, chargement des pages, expérience utilisateur.
- **Tests fonctionnels** : inscription, connexion, recherche, ajout au panier, commande.
- **Tests API** : cohérence des données reçues depuis le backend.

6. Performances et sécurité

6.1 Performances

- **Temps de réponse API** : optimisés grâce à PostgreSQL et à la structure REST.
- **Gestion du cache** : pour les données statiques (catégories, listes stables).
- **Optimisation des requêtes** : utilisation de `select_related` et `prefetch_related`.
- **Stabilité mobile** : Flutter assure de bonnes performances même sur appareils modestes.

6.2 Sécurité

- **Authentification JWT** : sécurisation des sessions utilisateur.
- **Communication HTTPS** : protection des échanges client-serveur.
- **Permissions DRF** : séparation stricte des rôles.
- **Validation serveur** : protection contre injections et données incorrectes.
- **Sécurisation des fichiers** : contrôle des accès aux ressources sensibles.

6.3 Lien GitHub

https://github.com/ka609/Projet_Tutore

7. Conclusion

Ce projet a permis de concevoir une solution complète répondant aux besoins de gestion pharmaceutique moderne. Les objectifs principaux ont été atteints.

Perspectives

- Paiement mobile intégré.
- Géolocalisation des pharmacies.
- Version Web complète.
- Notifications en temps réel.

Annexes

Diagrammes de conception



FIGURE 7.1 – Diagramme de cas d'utilisation général – Medishop

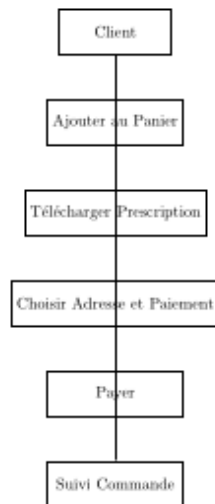


Figure 1: Diagramme de cas d'utilisation détaillé – Processus de commande Client

FIGURE 7.2 – Diagramme de cas d'utilisation détaillé – Processus de commande Client

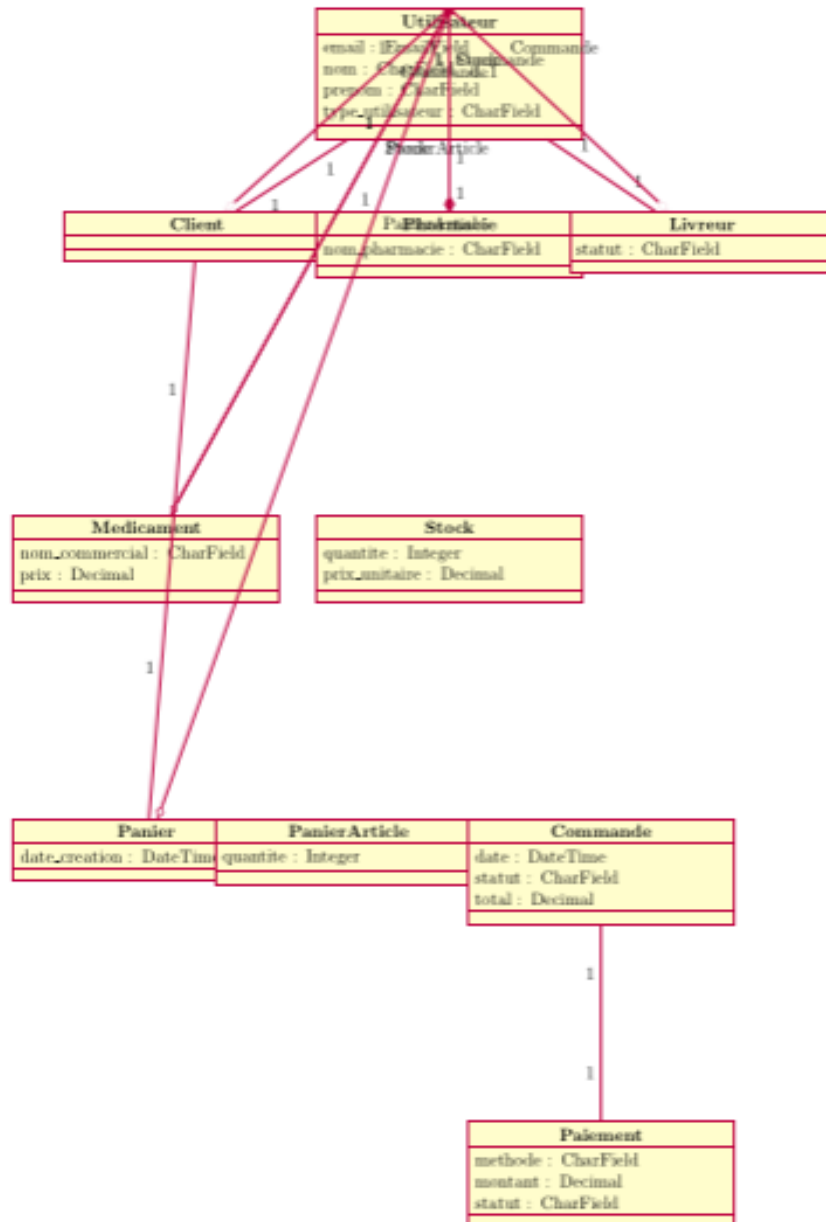


FIGURE 7.3 – Diagramme UML de classes – Medishop